



DevQR — The Complete Master Guide & Technical Manual

Laptop Errors. Phone Intelligence.

Zero DevQR cloud backend. Your code, stack traces, and debugging sessions belong to you.



Table of Contents

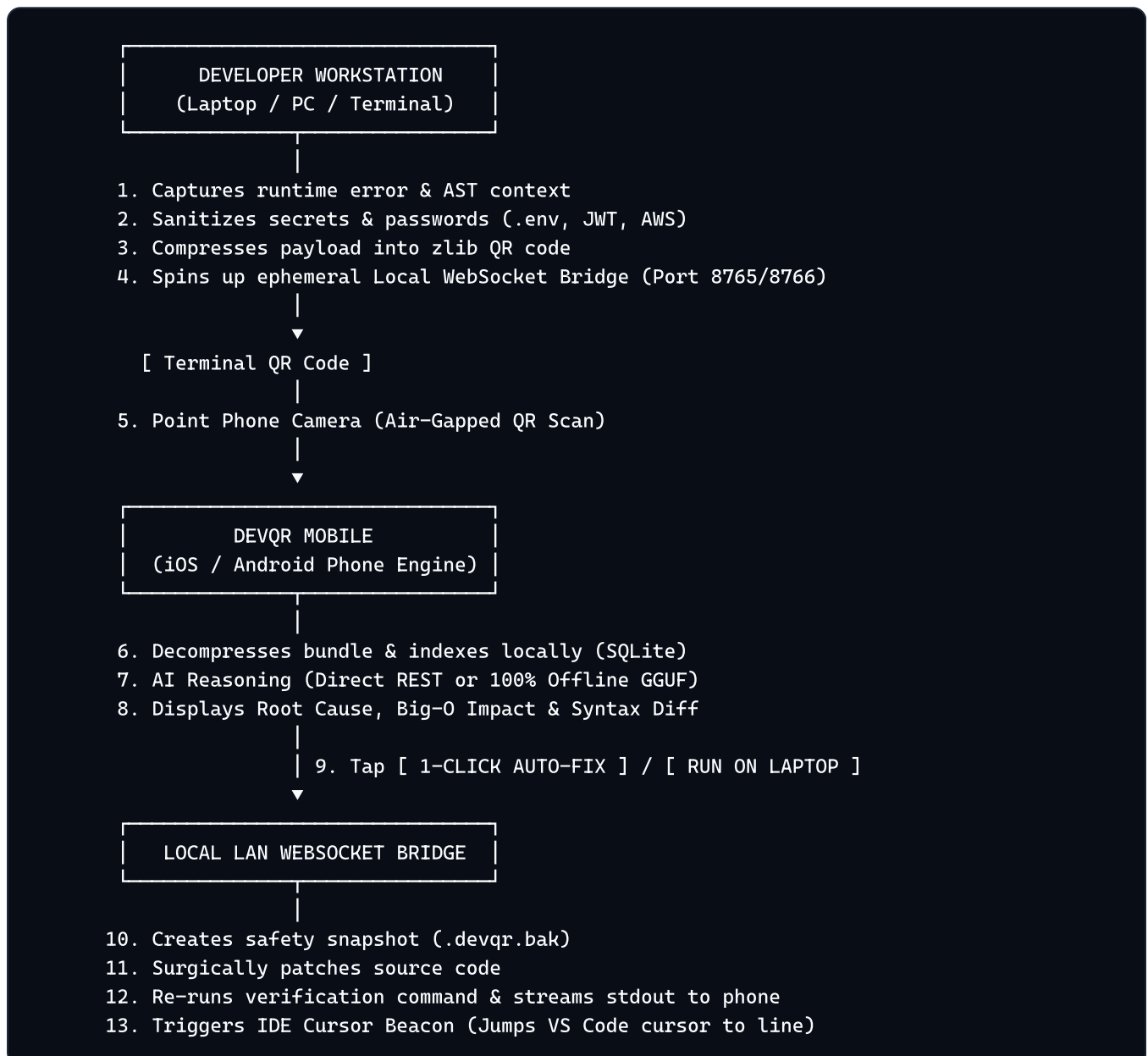
1. Architecture & How DevQR Works
2. Prerequisites & Complete Setup
 - CLI Setup on Laptop / PC
 - Mobile App Setup on Phone
 - AI Provider Configuration (Cloud & 100% Offline Local LLM)
3. CLI Commands Reference & Usage Guide
 - `devqr` / `devqr scan` (Standard Error Scanner)
 - `devqr -i` (Interactive Context Picker)
 - `devqr review <file>` (Single File Code Review & Audit)
 - `devqr gen` / `devqr new` (AI App & File Generator Studio)
 - `devqr term [cmd]` (Live Interactive Terminal Multiplexer)
 - `devqr arch` (Repository Architecture & Health Blueprint)
 - `devqr apply <patch>` (Offline Reverse Patch Transfer)
4. Mobile App Complete Feature Walkthrough
 - QR Scanner (`/index`)
 - AI Diagnostic & Repair Studio (`/result`)
 - Remote Laptop Actions (1-Click Auto-Fix, Undo, Remote Run, IDE Beacon)
 - Live Streaming Terminal (`/terminal`)
 - AI App & File Studio (`/generator`)
 - Architecture Studio & Health Scorecard (`/arch`)
 - Executive PDF Technical Exporter (Post-Mortem & Architecture)
 - Offline SQLite Sessions History (`/sessions`)
 - Settings & Local LLM Management (`/settings`)
5. Step-by-Step End-to-End Real World Workflows

- Workflow 1: 1-Click Fixing a Python / Node / Rust Bug
- Workflow 2: Full Architecture Audit & Exporting Executive PDF
- Workflow 3: Scaffolding a New Program from Phone onto Laptop
- Workflow 4: Remote Test Execution & Live Terminal Streaming
- Workflow 5: 100% Offline Airplane-Mode Debugging with Local LLM

6. Troubleshooting & FAQs

1. Architecture & How DevQR Works

DevQR is built with **zero external DevQR servers**. It creates a peer-to-peer connection between your computer and your phone:



2. Prerequisites & Complete Setup

2.1 CLI Setup on Laptop / PC

Prerequisites

- **Node.js:** v18.0.0 or higher
- **Git:** Installed and available in PATH
- **OS:** Windows, macOS, or Linux

Installation Steps

1. Open your terminal and navigate to the project directory:

```
cd d:/devqr
```

2. Install dependencies for the whole workspace:

```
npm install
```

3. Build and link the CLI globally:

```
cd cli  
npm install  
npm run build  
npm link
```

4. Verify CLI installation:

```
devqr --version  
# Output: 1.0.0
```

2.2 Mobile App Setup on Phone

Option A: Running via Expo Go (Fastest for Development)

1. Install **Expo Go** from Google Play Store or Apple App Store.
2. In your laptop terminal:

```
cd d:/devqr/mobile  
npm start
```

3. Open **Expo Go** on your phone and scan the Metro bundler QR code.

Option B: Installing the Standalone Production Android APK

- 1. Download the release APK from the GitHub Actions build (`DevQR-Android-APK`).
- 2. Install the `.apk` on your Android phone.
- 3. Enable "Install unknown apps" if prompted.

2.3 AI Provider Configuration

Open the **Settings** screen in DevQR Mobile (gear icon at bottom right):

Provider	Setup Requirement	Best For
 On-Device (GGUF)	No API Key needed. Tap <code>Download Qwen2.5-Coder</code> (986 MB).	100% Offline, Airplane mode, high security.
OpenRouter	Get API key from openrouter.ai .	DeepSeek-R1, Claude 3.5 Sonnet, GPT-4o, Llama 3.3.
Google Gemini	Free key from aistudio.google.com .	Ultra-fast live code reasoning (Flash / Pro).
Groq	Free key from console.groq.com .	Ultra-high speed LPU inference (500+ tokens/sec).
OpenAI	API key from platform.openai.com .	GPT-4o & GPT-4o-mini.
Anthropic	API key from console.anthropic.com .	Claude 3.5 Sonnet.

Privacy Guarantee: All API keys are encrypted inside your phone's hardware Keystore / Keychain. No keys or tokens are ever sent to DevQR.

3. CLI Commands Reference & Usage Guide

Usage: devqr [command] [options]

DevQR – Laptop Errors. Phone Intelligence. (Zero Cloud Backend)

Options:

<code>-v, --version</code>	output the version number
<code>-f, --file <path></code>	Single source file to review and inspect
<code>-e, --error <msg></code>	Error message or review specification
<code>-s, --stack <trace></code>	Stack trace
<code>-i, --interactive</code>	Interactively pick project files to include in QR payload
<code>-g, --generator</code>	Launch AI App & File Generator Studio
<code>--no-sanitize</code>	Skip sensitive information sanitization
<code>--json</code>	Output raw bundle JSON payload without QR graphic
<code>-h, --help</code>	display help for command

Commands:

<code>review check audit</code>	Perform a single file AI code review and bug detection
<code>gen new create</code>	Launch AI App & File Studio to build new code files from phone
<code>term repl shell run</code>	Launch live streaming terminal & interactive mobile REPL
<code>arch</code>	Scan repository architecture, dependency graph, and code health
<code>scan</code>	Scan project logs, git changes, and generate debugging QR
<code>collect</code>	Collect and sanitize debug context without rendering QR
<code>apply [payload]</code>	Apply an AI Fix Patch received from DevQR Mobile

3.1 `devqr` / `devqr scan`

Scans your workspace for recent crash logs, uncommitted git diffs, and project environment, then renders a high-density terminal QR code.

```
# Automatic scan in current directory
devqr

# Pass a specific runtime error message
devqr -e "TypeError: Cannot read property 'map' of undefined"

# Pass a specific source file and error
devqr -f src/server.ts -e "UnhandledPromiseRejection: Connection timeout at line 42"
```

3.2 `devqr -i` (Interactive Context Picker)

Launches an interactive terminal UI with arrow-key navigation to pick exactly which modified files and logs to include in the QR bundle:

```
devqr -i
```

- **Step 1:** Enter or edit the error message.
 - **Step 2:** Use Spacebar to select/deselect specific Git files.
 - **Step 3:** Toggle secret sanitization on/off.
 - **Step 4:** Terminal generates the customized QR code.
-

3.3 `devqr review <file>`

Perform an instant AI code review, architecture check, and security audit on any specific file:

```
# Review a Python script
devqr review app.py

# Review a TypeScript file with specific instructions
devqr review src/auth.ts -e "Check for SQL injection vulnerabilities and race conditions"
```

3.4 `devqr gen` / `devqr new`

Launches the **AI App & File Generator Studio** in any folder (new or existing):

```
mkdir my-new-project
cd my-new-project

# Launch generator
devqr gen
```

- Point phone at the QR code.
 - Choose a preset (*Number Guessing Game*, *CLI Todo*, *FastAPI Microservice*, *Terminal Snake*, etc.) or type custom prompt.
 - Tap `[ PUSH & OPEN IN IDE ]` to write the file directly into your laptop folder and open it in VS Code / Cursor!
-

3.5 `devqr term [cmd]` (Live Terminal Multiplexer)

Spins up a live bidirectional streaming terminal over local WebSocket. You can view logs and send inputs directly from your phone screen without generating any code.

```
# Open interactive terminal in current directory
devqr term

# Run a specific script and stream to phone
devqr term python game.py

# Run test suite and stream live results
devqr term npm test
```

3.6 `devqr arch`

Scans the entire repository AST, import relationships, and file sizes to create an architecture map:

```
devqr arch
```

- Scans up to 100+ files across the repository.
- Generates dependency graph and module tree.
- Displays System Blueprint, File Responsibilities, End-to-End Data Pipeline, Dead Code, and Technical Debt score on phone.

3.7 `devqr apply <patch>`

Applies a surgical code patch received from DevQR Mobile in air-gapped environments:

```
# Apply a copied patch code
devqr apply "devqr://patch/ ... "

# Or via stdin
cat patch.txt | devqr apply
```

4. Mobile App Complete Feature Walkthrough

4.1 QR Scanner Screen (`/index`)

- **Instant Auto-Scan:** High-speed QR scanner with active target box and laser animation.
- **Multi-Chunk Reassembly:** Automatically reassembles multi-part QR codes for large codebases.
- **Recent Sessions Bar:** Quick-tap past sessions stored in local SQLite storage.

4.2 AI Diagnostic & Repair Studio (`/result`)

When a QR code is scanned, DevQR executes deep multi-layer analysis:

1. **Root Cause Analysis:** Explains exactly why the error occurred with code context.
 2. **Algorithmic Big-O Profiler:** Compares Time and Space complexity before vs after the fix.
 3. **Interactive Syntax Diff:**
 - **Unified Diff view** with red deletions and green additions.
 - **Full Original file** view.
 - **Full Patched file** view.
 4. **Smart Action Chips:**
 - `Generate Unit Test` : Synthesizes regression test cases.
 - `Security Audit` : Scans for SQLi, XSS, prototype pollution, memory leaks.
 - `Explain for Junior Dev` : Generates simple, analogy-based explanations.
-

4.3 Remote Laptop Actions

When connected via LAN Bridge:

- `[ 1-CLICK AUTO-FIX ]` : Laptop creates `.devqr.bak` safety snapshot and applies patch in real-time.
 - `[ Undo / Revert ]` : Instantly restores the original file if needed.
 - `[ Run & Verify on Laptop ]` : Executes verification command (`python app.py` , `npm test`) on laptop and streams output to phone.
 - `[ Focus in IDE ]` : Teleports cursor in VS Code, Cursor, or Neovim directly to the exact file and line number.
 - `[ Write Test File to Laptop ]` : Automatically writes a new unit test file (`test_fix.py` , `fix.test.ts`) into laptop workspace and runs it.
-

4.4 Live Streaming Terminal (`/terminal`)

- **ANSI Color Stream:** Real-time terminal output rendering.
 - **Interactive Mobile Keyboard:** Send text, numbers, menu choices, or `Enter` keystrokes to running laptop terminal.
 - **Quick Action Controls:** Stop process (`Ctrl+C`), Clear screen, or Re-run command.
-

4.5 AI App & File Studio (`/generator`)

- **Natural Language Scaffolding:** Describe any application you want to build.

- **Built-in Presets:**
 - Terminal Snake Game (Python / Node)
 - Number Guessing Game with Leaderboard
 - CLI Todo Manager with JSON persistence
 - FastAPI Microservice
 - CSV Data Analyzer
 - **1-Click Push:** Writes generated file to laptop workspace and opens in editor.
 - **1-Click Run:** Executes script on laptop immediately and opens streaming terminal.
-

4.6 Architecture Studio (`/arch`)

- **System Blueprint & Pattern:** Detects MVC, Monorepo, Microservice, Layered Architecture.
 - **Maintainability Grade:** Scorecard (`Grade: A/B/C`) with estimated Technical Debt hours.
 - **Module Hierarchy Table:** Purpose and responsibility breakdown per file.
 - **End-to-End Request Pipeline:** Step-by-step data lifecycle tracing.
 - **Dead Code & Anti-Pattern Auditor:** Highlights unused modules and circular dependencies.
-

4.7 Executive PDF Exporters

DevQR generates styled A4 PDF documents directly from your phone:

1. **Architecture Audit PDF:** Complete blueprint, table of components, debt analysis.
 2. **Incident Post-Mortem PDF:** Incident error, root cause, code patch diff, and verification checklist.
 3. **Native Sharing:** Instantly share via AirDrop, Email, Slack, WhatsApp, or Print.
-

4.8 Offline SQLite Sessions History (`/sessions`)

- **Zero Cloud Storage:** All debugging history is stored locally in phone SQLite database.
 - **Filter & Search:** Search past sessions by error message, project name, or language.
 - **Re-run & Review:** Open any past session offline without re-scanning.
-

4.9 Settings & Offline Model Manager (`/settings`)

- Switch AI providers with one tap.
 - Encrypted API key management.
 - Download, verify, and delete on-device **Qwen2.5-Coder (986 MB)** GGUF model.
-

5. Step-by-Step End-to-End Workflows

Workflow 1: 1-Click Fixing a Bug

1. On Laptop, run:

```
devqr
```

2. Open **DevQR Mobile** on your phone and scan the terminal QR code.
 3. Review the root cause and syntax diff.
 4. Tap **[1-CLICK AUTO FIX (ON LAPTOP)]**.
 5. Laptop automatically creates `.devqr.bak` and patches the file.
 6. Tap **[Run & Verify on Laptop]** to confirm the fix works.
-

Workflow 2: Architecture Audit & PDF Export

1. On Laptop, run in project root:

```
devqr arch
```

2. Scan QR code with phone.
 3. Inspect system architecture blueprint, module responsibilities, and technical debt score.
 4. Tap **[Export Architecture PDF]** to generate and share an executive report.
-

Workflow 3: Scaffolding a New Program

1. On Laptop:

```
mkdir cli-tool && cd cli-tool  
devqr gen
```

2. Scan QR code with phone.
 3. Select a preset or type: *"Create an interactive budget manager in Python"*.
 4. Tap **[Generate Code]**.
 5. Tap **[PUSH & OPEN IN IDE]** -> file appears in VS Code.
 6. Tap **[RUN ON LAPTOP]** -> interact with the program from phone!
-

Workflow 4: Remote Test Execution

1. On Laptop:

```
devqr term npm test
```

2. Scan QR code with phone.
 3. Watch test suite execute in real-time on your phone screen with live ANSI colors.
-

Workflow 5: 100% Offline Airplane-Mode Debugging

1. In DevQR Mobile -> **Settings** -> tap  **On-Device (GGUF)**.
 2. Tap  **Download Qwen2.5-Coder (986 MB)** (one-time setup).
 3. Put phone into **Airplane Mode** (turn off Wi-Fi and Mobile Data).
 4. Scan any error QR code with phone.
 5. DevQR runs the local neural network directly on your phone's processor to diagnose and generate fixes with 0 internet connection.
-

6. Troubleshooting & FAQs

Q: Why does the phone fail to connect to the laptop LAN bridge?

- Ensure both your laptop and phone are connected to the **same Wi-Fi network**.
- If on a guest or corporate Wi-Fi with client isolation enabled, connect your phone via Mobile Hotspot or USB tethering.
- Ensure firewall allows local connections on ports `8765` and `8766`.

Q: Are my secrets, tokens, or environment variables exposed in QR codes?

- **No.** DevQR runs a mandatory regex & high-entropy secret scrubbing engine (`sanitizer.ts`) before bundle creation. AWS keys, JWTs, `.env` passwords, and bearer tokens are automatically sanitized into `[REDACTED]`.

Q: How do I undo a patch applied by DevQR?

- Tap `[ Undo / Revert ]` on your phone screen, or on your laptop restore the automatic `.devqr.bak` snapshot file created alongside your source file.
-